

Evolución de la clase ProbNet

Del diagnóstico al rediseño: proceso, decisiones y lecciones

org.openmarkov.core — paquete model.network

Manuel Arias

Abril de 2026

Índice

1. Fundamentos de ingeniería del software	3
1.1. Principios de diseño	3
1.2. Antipatrones detectados	3
1.3. Patrones de diseño aplicados	4
2. El punto de partida: por qué ProbNet necesitaba cambiar	4
2.1. Una clase de importancia crítica	4
2.2. La herencia directa de Graph	4
2.3. La acumulación de deuda técnica	5
3. El diagnóstico	5
3.1. Los bugs confirmados	6
3.1.1. B1: getChanceAndDecisionVariables() incluye tipos extra	7
3.1.2. B2: moveNode() — NullPointerException potencial	7
3.1.3. B3: modifyAgent(REMOVE) deja agents = null	7
3.1.4. B4: addShiftedNode() no copia potenciales	8
3.2. Los code smells	8
3.2.1. S1: Lógica duplicada entre checkProbNet() y getUnsatisfiedConstraints() (violación DRY)	8
3.2.2. S2: Bucles manuales en clasificación de red	8
3.2.3. S3: Patrón duplicado entre getProbPotentials() y getUtilityPotentials() (violación DRY)	9
3.2.4. S4: getNumCriteria() con complejidad $O(n \cdot m)$	9
3.2.5. S5: Código comentado e if-chain en addNodeConsistently()	9
3.2.6. S6: toString() en la línea 53	9
3.2.7. S7: reader/writer en el modelo de dominio (violación SRP y DIP)	9
3.3. Cobertura de tests	10
3.3.1. Resumen cuantitativo	12
4. La hoja de ruta: diseñar el cambio antes de ejecutarlo	12
4.1. El principio rector: retrocompatibilidad total	12
4.2. La arquitectura objetivo	13
4.3. Fase 0: ampliar los tests (prerrequisito)	14
4.4. Las fases y su riesgo estimado	14
5. La ejecución del rediseño	14

5.1.	El estado real de partida	14
5.2.	Fase 1: corregir bugs y simplificaciones	16
5.3.	Extracción de <code>NetworkMetadata</code>	16
5.4.	Extracción de <code>ProbNetCopier</code>	18
5.5.	Extracción de <code>ProbNetClassifier</code>	18
5.6.	Extracción de <code>ProbNetAgentManager</code>	19
5.7.	Extracción de <code>ProbNetPotentialQueries</code>	20
5.8.	El cambio más profundo: de herencia a composición	21
6.	El resultado	24
6.1.	La nueva arquitectura	24
6.2.	Las clases extraídas	25
6.3.	Métricas comparadas	25
6.4.	Estado de ejecución de las fases	25
6.5.	Qué no cambió: la API pública	26
7.	Reflexión sobre el proceso	26
7.1.	Lo que facilitó la refactorización	26
7.2.	Lo que fue más difícil de lo esperado	26
7.3.	Lecciones para futuros procesos de refactorización	27

1. Fundamentos de ingeniería del software

Esta sección define los conceptos de ingeniería del software que se aplican a lo largo del documento. Es especialmente útil para lectores con poca experiencia en diseño orientado a objetos: los términos técnicos se usan con precisión en el resto del texto.

1.1. Principios de diseño

Los *principios de diseño* son reglas generales que, cuando se aplican con criterio, producen código más fácil de entender, modificar y probar. No son leyes inmutables: su valor está en que obligan a *justificar* por escrito las decisiones de diseño.

Principio de Responsabilidad Única (SRP). Una clase debe tener **una sola razón para cambiar** [2]. Si un módulo hace demasiadas cosas a la vez, cualquier modificación en una de ellas obliga a revisar y re-probar todas las demás. El indicador más claro de que una clase viola el SRP es que necesita importaciones de paquetes que “no le corresponden”: si el modelo de dominio importa clases de I/O, algo va mal.

Principio DRY (*Don't Repeat Yourself*). Cada pieza de conocimiento debe tener **una única representación** en el sistema [4]. Cuando la misma lógica aparece en dos lugares, una corrección debe aplicarse en ambos; si se olvida uno, el comportamiento queda inconsistente.

Principio de Inversión de Dependencias (DIP). Los módulos de alto nivel (el modelo de dominio) **no deben depender de los módulos de bajo nivel** (I/O, persistencia) [2]. Ambos deben depender de abstracciones. Si el modelo sabe cómo leer y escribir ficheros, un cambio de formato puede obligar a tocar la parte más delicada del sistema.

Composición frente a herencia. Existen dos formas de reutilizar comportamiento en programación orientada a objetos: la *herencia* (“B es un tipo de A”) y la *composición* (“B tiene un objeto de tipo A”). La herencia crea un acoplamiento muy fuerte: la subclase hereda *todo* lo del padre, incluyendo lo que no necesita, y cualquier cambio en el padre puede romper la subclase. Un principio clásico recomienda **preferir la composición** siempre que la relación no sea genuinamente “es un tipo de” [3].

Principio de Segregación de Interfaces (ISP). Una clase no debe ser obligada a depender de métodos que no utiliza. Es preferible tener **varias interfaces pequeñas y específicas** que una sola interfaz grande [2]: así, cada consumidor declara exactamente las capacidades que necesita. Un algoritmo de inferencia que solo consulta el grafo no debería necesitar importar una clase con 80 métodos.

1.2. Antipatrones detectados

Un *antipatrón* es un patrón de diseño que, en apariencia razonable, genera en la práctica problemas de mantenimiento. Reconocerlos en el propio código es la mitad del trabajo.

God Class (Clase Dios). Una clase que acumula demasiadas responsabilidades: lo sabe todo y lo hace todo [1]. Sus síntomas son: cientos o miles de líneas, métodos de dominios no relacionados y muchas dependencias entrantes (todo el sistema pasa por ella). Es dañina porque viola el SRP, hace imposibles las pruebas unitarias aisladas y se convierte en el cuello de botella del desarrollo.

Herencia inapropiada. Usar herencia para reutilizar la implementación de otra clase aunque la relación “es un tipo de” no sea semánticamente correcta. El resultado es que la subclase hereda métodos y campos que no le corresponden y queda atada a todos los cambios del padre. La solución es sustituir la herencia por composición y declarar las capacidades mediante interfaces.

1.3. Patrones de diseño aplicados

Extract Class (Extraer Clase). Cuando una clase tiene demasiadas responsabilidades, se identifica un grupo cohesivo de métodos y campos y se mueve a una nueva clase [1]. La clase original delega en la nueva. Este patrón no cambia el comportamiento externo: si se aplica correctamente, los tests existentes siguen pasando sin modificación alguna.

Value Object (Objeto Valor). Un objeto que agrupa datos relacionados sin identidad propia: lo que importa es su contenido, no su dirección en memoria. Útil para separar los datos de configuración del comportamiento de dominio.

Interfaces como contrato mínimo. En lugar de hacer que todo el código cliente dependa de una clase concreta grande, se definen interfaces que describen solo las capacidades que cada cliente necesita. La clase concreta implementa todas las interfaces aplicables. El cliente declara su dependencia sobre la interfaz: si mañana cambia la implementación, el cliente no necesita modificarse.

2. El punto de partida: por qué ProbNet necesitaba cambiar

2.1. Una clase de importancia crítica

ProbNet es la clase más referenciada de OpenMarkov. **490 ficheros** la importan en alguna de sus formas. Representa la red probabilística en sí misma: sus nodos, arcos, potenciales, restricciones y metadatos. Sin ella, el proyecto no existe.

Precisamente por su centralidad, ProbNet fue acumulando responsabilidades durante años. En el momento del análisis, la clase tenía **1440 líneas, más de 80 métodos públicos** y cubría seis áreas de responsabilidad completamente distintas:

1. Gestión de nodos, arcos y variables.
2. Gestión de potenciales (tablas de probabilidad y utilidad).
3. Restricciones de la red.
4. Clasificación de la red (¿es temporal? ¿multiagente? ¿solo nodos chance?).
5. Gestión de agentes.
6. Metadatos descriptivos e I/O (nombre, comentario, lector/escritor de ficheros).

Cada una de estas áreas tiene su propia razón para cambiar. Un bug en la gestión de agentes no tiene nada que ver con la lógica de potenciales, pero en el mismo fichero ambas convivían. Esto es la definición de God Class (§1.2).

2.2. La herencia directa de Graph

ProbNet extendía `Graph<Node>`, la clase genérica de grafo de la librería interna. Esta elección de diseño tuvo sentido en su momento, pero con el tiempo generó dos problemas:

- ProbNet exponía la API completa de `Graph`: métodos como `addVertex()` y `removeVertex()` (operaciones de bajo nivel del grafo genérico) formaban parte del contrato público de la red probabilística. No había forma de distinguir las operaciones de dominio de las de infraestructura.

- No existía ninguna interfaz que permitiera a un algoritmo declarar que solo necesitaba *consultar* la red. Todo el código que quería consultar la topología debía aceptar **ProbNet** completa, con sus 80 métodos, aunque solo usase tres. Esto viola el ISP y el DIP (§1.1).

2.3. La acumulación de deuda técnica

La situación no era el resultado de malas decisiones puntuales, sino de una acumulación gradual de deuda técnica. Cada nueva funcionalidad se añadía donde más cómodo resultaba — en **ProbNet**, porque ya tenía acceso a todo. Esta es la dinámica habitual de las God Classes: no nacen grandes; crecen.

Al momento del análisis, la deuda era cuantificable:

Métrica	Valor
Líneas de código	1 440
Métodos públicos	80+
Campos	16
Ficheros dependientes	490
Áreas de responsabilidad	6
Bugs conocidos	4
Code smells documentados	7
Cobertura de tests (métodos)	37 %

3. El diagnóstico

El primer paso fue documentar con precisión la situación existente, sin empezar a cambiar nada todavía. La Figura 1 muestra el diagrama resultante.

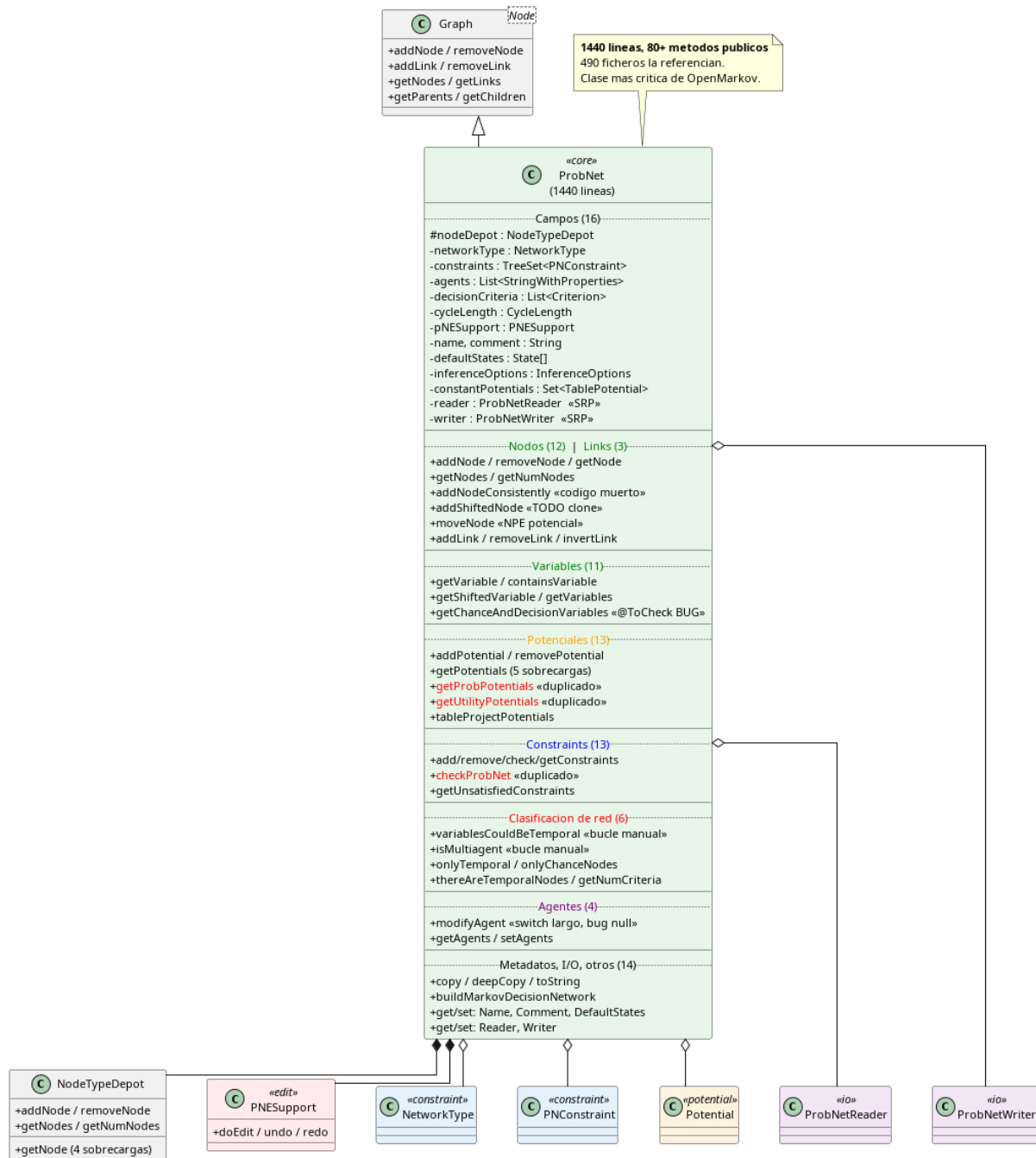


Figura 1: Diagrama UML de ProbNet antes del rediseño. Los colores agrupan las seis áreas de responsabilidad; las etiquetas en rojo señalan los problemas.

ProbNet extendía `Graph<Node>` y añadía:

- **16 campos:** desde `nodeDepot` (indexación de nodos por tipo) hasta `reader/writer` (I/O).
- **80+ métodos públicos:** gestión de nodos (12), links (3), variables (11), potenciales (13), constraints (13), clasificación de red (6), agentes (4), metadatos e I/O (12), y otros (copia, inferencia, criterios).

3.1. Los bugs confirmados

La Figura 2 resume los bugs, code smells y riesgos identificados.

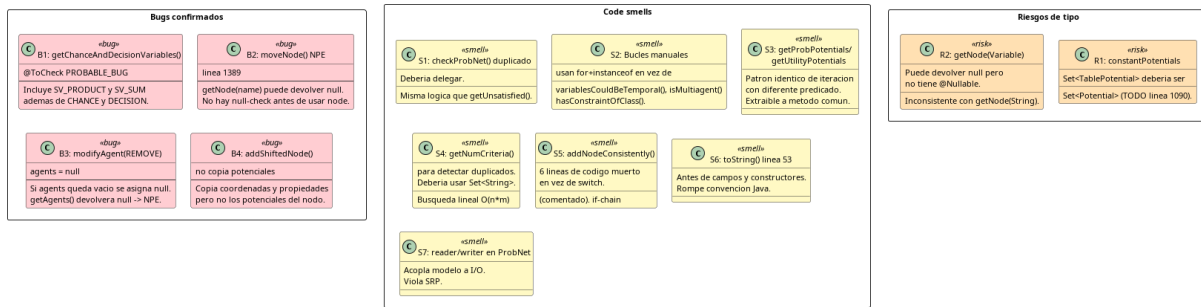


Figura 2: Bugs confirmados (rojo), code smells (amarillo) y riesgos de tipo (naranja) en ProbNet.

3.1.1. B1: getChanceAndDecisionVariables() incluye tipos extra

El método está anotado con @ToCheck(PROBABLE_BUG). Su nombre indica que devuelve solo variables de tipo CHANCE y DECISION, pero el switch también incluye SV_PRODUCT y SV_SUM:

```
1 case CHANCE , SV_PRODUCT , SV_SUM , DECISION -> true;
2 case UTILITY -> false;
```

Listado 1: B1: el switch incluye tipos no mencionados en el nombre.

Si el comportamiento actual es intencionado, el nombre del método es engañoso. Si no lo es, devuelve variables de más.

3.1.2. B2: moveNode() — NullPointerException potencial

En la línea 1389, getNode(name) puede devolver null si el nombre no existe en la red. No hay comprobación de nulidad antes de usar el resultado:

```
1 Node node = getNode(name);
2 // NPE aquí si name no existe en la red:
3 node.setCoordinateX(newPositions.get(i).getX());
4 node.setCoordinateY(newPositions.get(i).getY());
```

Listado 2: B2: acceso sin null-check sobre el resultado de getNode().

3.1.3. B3: modifyAgent(REMOVE) deja agents = null

Cuando se elimina el último agente, el método asigna agents = null:

```
1 case REMOVE -> {
2     for (int i = 0; i < agents.size(); i++) {
3         if (agents.get(i).getString().equals(agentName)) {
4             agents.remove(i);
5         }
6     }
7     if (agents.size() == 0) {
8         agents = null; // <-- cualquier llamada posterior a
9         //           getAgents() causa NPE
10    }
11 }
```

Listado 3: B3: la asignación agents = null causa NPE posterior.

getAgents() devuelve el campo directamente sin comprobar nulidad, por lo que cualquier iteración posterior lanza NullPointerException. Además, modifyAgent(ADD) crea la lista si es null, pero nunca debería haberlo sido.

3.1.4. B4: addShiftedNode() no copia potenciales

El método copia el tipo de nodo, coordenadas, propósito, relevancia, comentario y propiedades adicionales, pero **no copia los potenciales** del nodo original ni crea uno por defecto. El propio código reconoce el problema con un comentario TODO:

```
1 // TODO Hacer clon para node y quitar estas lineas
2 node.setNodeType(originalNode.getNodeType());
3 node.setCoordinateX(originalNode.getCoordinateX() + shiftX);
4 node.setCoordinateY(originalNode.getCoordinateY() + shiftY);
5 node.setPurpose(originalNode.getPurpose());
6 // ... propiedades copiadas ...
7 // Potenciales: NO se copian (omision confirmada)
```

Listado 4: B4: potenciales no copiados; el TODO reconoce la deuda.

3.2. Los code smells

3.2.1. S1: Lógica duplicada entre checkProbNet() y getUnsatisfiedConstraints() (violación DRY)

Ambos métodos iteraban sobre `constraints` con la misma condición `constraint.isMetBy(this)`. Si la lógica de verificación cambiaba, había que actualizar dos métodos:

```
1 public boolean checkProbNet() {
2     for (PNConstraint constraint : constraints) {
3         if (!constraint.isMetBy(this)) { return false; }
4     }
5     return true;
6 }
7
8 public List<PNConstraint> getUnsatisfiedConstraints() {
9     List<PNConstraint> unsatisfied = new ArrayList<>();
10    for (PNConstraint constraint : constraints) {
11        if (!constraint.isMetBy(this)) {
12            unsatisfied.add(constraint);
13        }
14    }
15    return unsatisfied;
16 }
```

Listado 5: S1: el mismo bucle aparece en dos métodos (violación DRY).

3.2.2. S2: Bucles manuales en clasificación de red

`variablesCouldBeTemporal()` e `isMultiagent()` usaban un bucle `for` manual sobre `constraints` con `instanceof`, cuando ya existía `hasConstraintOfClass()` que hace exactamente eso:

```
1 // Antes (7 lineas con logica que hasConstraintOfClass() ya encapsula):
2 public boolean variablesCouldBeTemporal() {
3     for (PNConstraint constraint : constraints) {
4         if (constraint instanceof OnlyAtemporalVariables) return false;
5     }
6     return true;
7 }
8
9 // Despues (1 linea, aprovechando el metodo existente):
10 public boolean variablesCouldBeTemporal() {
11     return !hasConstraintOfClass(OnlyAtemporalVariables.class);
12 }
```

Listado 6: S2: bucle manual donde existe un método más expresivo.

3.2.3. S3: Patrón duplicado entre `getProbPotentials()` y `getUtilityPotentials()` (violación DRY)

Ambos métodos seguían el mismo algoritmo: obtener el nodo de la variable, reunir vecinos, iterar potenciales y filtrar por rol. Solo difería el predicado de filtro (CONDITIONAL_PROBABILITY frente a UTILITY). Con 40 líneas de código duplicadas, un error en el algoritmo de búsqueda debería corregirse en dos sitios.

3.2.4. S4: `getNumCriteria()` con complejidad $O(n \cdot m)$

El método usaba una `List<String>` con búsqueda lineal para detectar duplicados:

```
1 // Antes: List + contains() = O(n*m)
2 List<String> criteriaNames = new ArrayList<>();
3 for (Potential potential : getPotentials(PotentialRole.UTILITY)) {
4     String name = potential.getCriterion().toString();
5     if (!criteriaNames.contains(name)) { // búsqueda lineal
6         criteriaNames.add(name);
7     }
8 }
9 return criteriaNames.size();
10
11 // Despues: Set = O(n log n)
12 Set<String> criteriaNames = new TreeSet<>();
13 getPotentials(PotentialRole.UTILITY).forEach(
14     p -> criteriaNames.add(p.getCriterion().toString());
15 return criteriaNames.size();
```

Listado 7: S4: complejidad cuadrática innecesaria con `List`.

3.2.5. S5: Código comentado e `if-chain` en `addNodeConsistently()`

Las líneas 933–938 contenían 6 líneas de código comentado que nunca se ejecutaban. Además, el método usaba una cadena de `if` en lugar de `switch`, lo que dificulta la lectura cuando hay más de tres ramas.

3.2.6. S6: `toString()` en la línea 53

El método `toString()` estaba ubicado al inicio del fichero, antes incluso de la declaración de los campos. La convención en Java es: campos → constructores → métodos de instancia → métodos auxiliares. Colocar `toString()` al principio sugiere que fue añadido puntualmente para depuración y nunca se reubicó.

3.2.7. S7: `reader/writer` en el modelo de dominio (violación SRP y DIP)

`ProbNet` almacenaba referencias a `ProbNetReader` y `ProbNetWriter`, los objetos responsables de la lectura y escritura de ficheros en disco. El motivo era funcional: la GUI necesitaba saber con qué formato se había abierto la red para usar el mismo formato al guardar. Sin embargo, esa información pertenece al controlador de ficheros, no al modelo de dominio. Esta violación del DIP acoplaba el módulo `core` con los módulos `io` y `gui`.

3.3. Cobertura de tests

Las Figuras 3, 4 y 5 muestran el estado de la cobertura de tests por área funcional.



Figura 3: Cobertura de tests: nodos (64 %), links (67 %) y variables (36 %).



Figura 4: Cobertura de tests: potenciales (56 %), constraints (31 %) y copia (100 %).

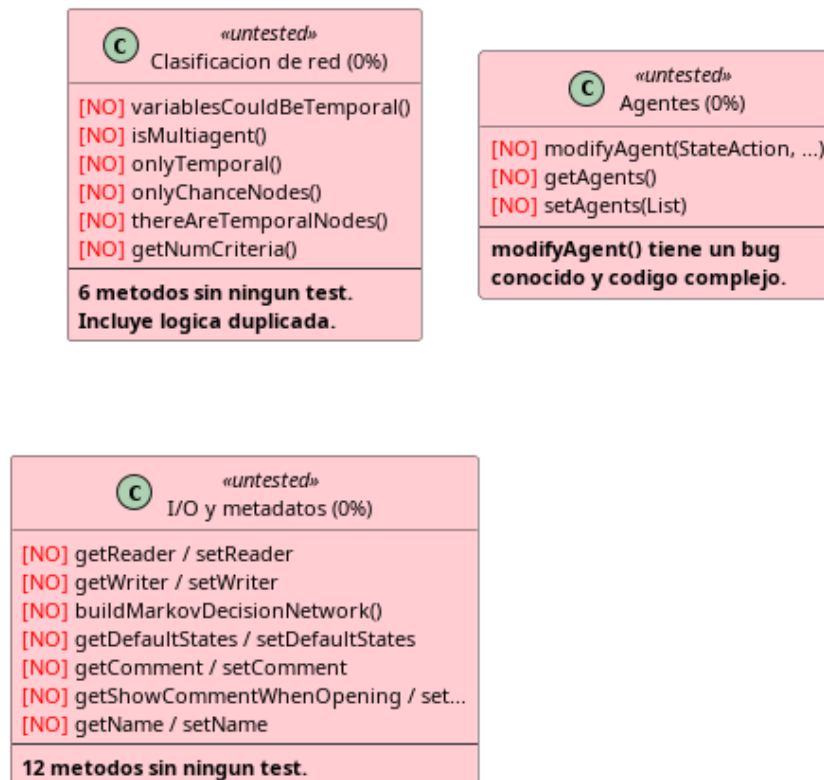


Figura 5: Cobertura de tests: clasificación de red (0 %), agentes (0 %) e I/O y metadatos (0 %). Estas tres áreas eran las lagunas más críticas.

3.3.1. Resumen cuantitativo

Área	Métodos	Testados	Cobertura
Gestión de nodos	14	9	64 %
Gestión de links	3	2	67 %
Variables	11	4	36 %
Potenciales	16	9	56 %
Constraints	13	4	31 %
Clasificación de red	6	0	0 %
Agentes	4	0	0 %
Copiar	2	2	100 %
I/O y metadatos	12	0	0 %
Total	81	30	37 %

Existían **30 tests** en ProbNetTest y **30 tests** en ProbNetCopyTest, pero **51 métodos públicos** no tenían ningún test unitario directo.

La ausencia total de tests en clasificación, agentes y metadatos no es una coincidencia: son exactamente las áreas donde vivían los bugs B1, B3 y el smell S7. Hay una relación circular entre God Class y ausencia de tests: una clase grande con muchas responsabilidades es difícil de aislar para probar, y sin tests la cobertura baja, lo que hace más difícil detectar bugs y más arriesgado refactorizar.

4. La hoja de ruta: diseñar el cambio antes de ejecutarlo

4.1. El principio rector: retrocompatibilidad total

Con 490 ficheros dependientes, la tentación es hacer un rediseño “de una sola vez” que resuelva todo al mismo tiempo. Esa estrategia es arriesgada: cualquier error rompe todo y puede ser difícil de localizar.

Se adoptó el principio de que **cada fase debe dejar el sistema funcionando**: al final de cada fase, todos los tests deben pasar. Esto convierte el proceso en una sucesión de pasos seguros, cada uno verificable de forma independiente.

La técnica que hace posible este principio es la **delegación retrocompatible**: cuando se extrae un método a una clase nueva, no se elimina de ProbNet. En su lugar, el método original se convierte en una llamada a la nueva clase:

```
1 // ProbNet.java ANTES:
2 public boolean isMultiagent() {
3     for (PNConstraint constraint : constraints) {
4         if (constraint instanceof OnlyOneAgent) return false;
5     }
6     return true;
7 }
8
9 // ProbNet.java DESPUES de extraer ProbNetClassifier:
10 public boolean isMultiagent() {
11     return ProbNetClassifier.isMultiagent(this); // delega
12 }
```

Listado 8: Patrón de delegación retrocompatible.

El código que llama a `probNet.isMultiagent()` **no necesita cambiar nada**. La refactorización es invisible para los consumidores de la clase.

4.2. La arquitectura objetivo

La Figura 6 muestra el objetivo planteado al final del análisis.

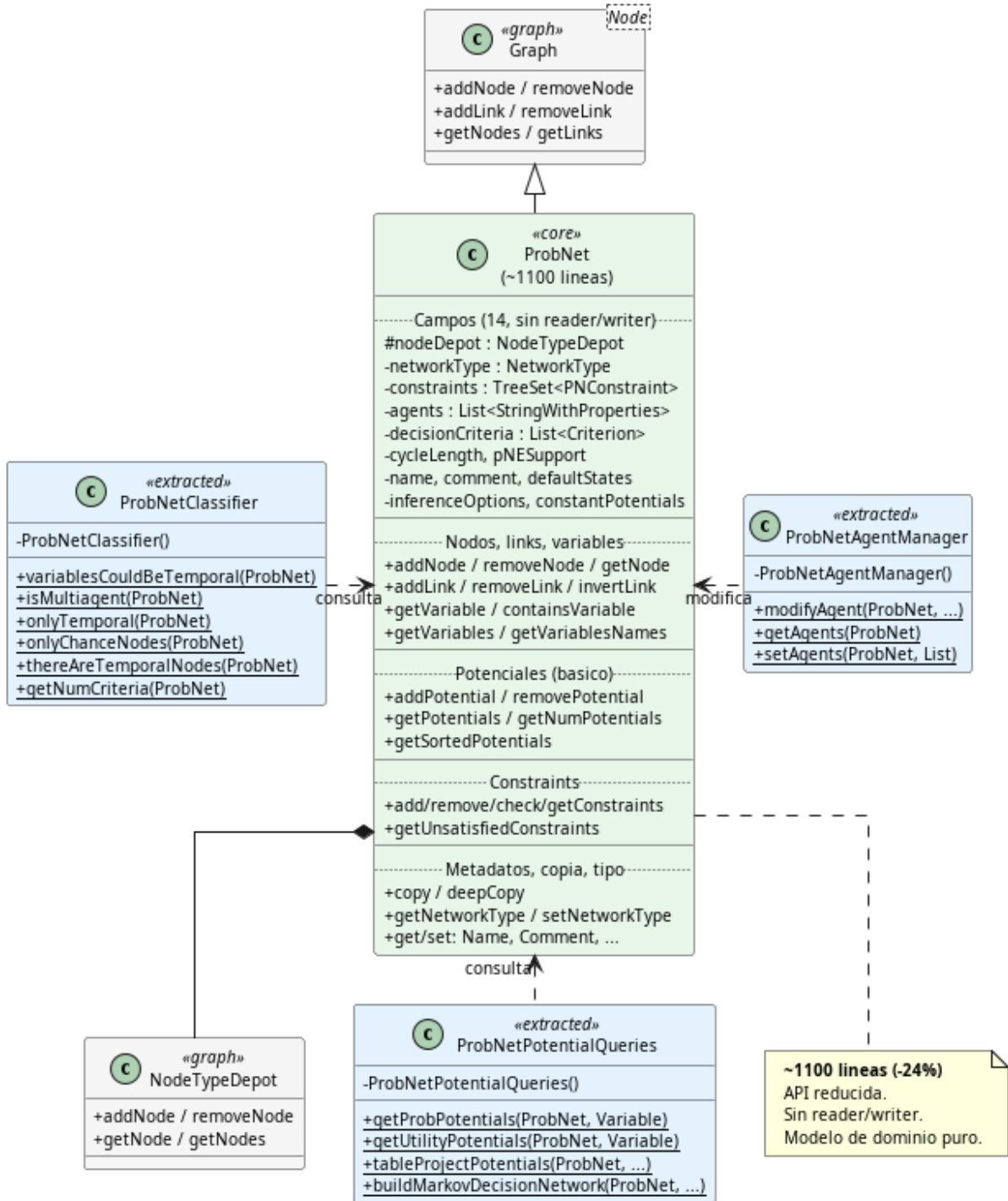


Figura 6: Arquitectura propuesta al inicio del proceso. Se identificaron tres extracciones principales; la ejecución añadió dos más.

La propuesta inicial identificaba tres grupos claros para extracción: clasificación de la red, gestión de agentes y consultas de potenciales. Durante la ejecución se añadieron dos más (NetworkMetadata y ProbNetCopier) y se ejecutó el cambio más profundo: la sustitución de la herencia por composición con interfaces.

4.3. Fase 0: ampliar los tests (prerrequisito)

Antes de cualquier refactorización, era necesario cubrir las lagunas críticas de cobertura. Sin tests, la refactorización es potencialmente destructiva; con tests, es un proceso predecible.

Área	Tests a añadir	Prioridad
Variables	getVariable, containsVariable, getShiftedVariable, getVariablesNames, updateVariable	Alta
Constraints	setNetworkType (cambio válido e inválido con roll-back), checkProbNet, getUnsatisfiedConstraints, hasConstraintOfClass	Alta
Clasificación	Todos los 6 métodos con redes BN, ID y DBN	Alta
Agentes	modifyAgent ADD/REMOVE/RENAME con casos límite	Alta
Potenciales	addPotential(pot, origNet), getProbPotentials, getSortedPotentials, getConstantPotentials	Media
Nodos	moveNode (normal y nombre inválido), addShiftedNode, invertLink	Media
Metadatos	getDefaultStates, comment, showCommentWhenOpening	Baja

Objetivo: pasar de **37 %** a **>80 %** de cobertura de métodos públicos antes de empezar las extracciones.

4.4. Las fases y su riesgo estimado

Fase	Descripción	Riesgo	Dependencia
0	Ampliar tests	Bajo	—
1	Corregir bugs y code smells	Bajo	Fase 0
	Extraer NetworkMetadata	Bajo	Fase 0
	Extraer ProbNetCopier	Bajo	Fase 0
3	Extraer ProbNetClassifier	Medio	Fase 0, 1
4	Extraer ProbNetAgentManager	Bajo	Fase 0, 1
5	Extraer ProbNetPotentialQueries	Medio	Fase 0
	Herencia → composición + interfaces	Alto	Fase 0
6	Evaluar mover reader/writer a la GUI	Alto	Fase 0

Las fases 0–1 son de bajo riesgo y alto valor. Las fases 3–5 siguen el patrón *Extract Class* ya probado con éxito en `VariableStateOperations` y `VariableTypeConverter`. La fase 6 requiere un análisis de impacto en los módulos `gui` e `io`.

5. La ejecución del rediseño

5.1. El estado real de partida

La Figura 7 muestra la estructura interna de `ProbNet` agrupada por área de responsabilidad, con el color de cada sección indicando el dominio.

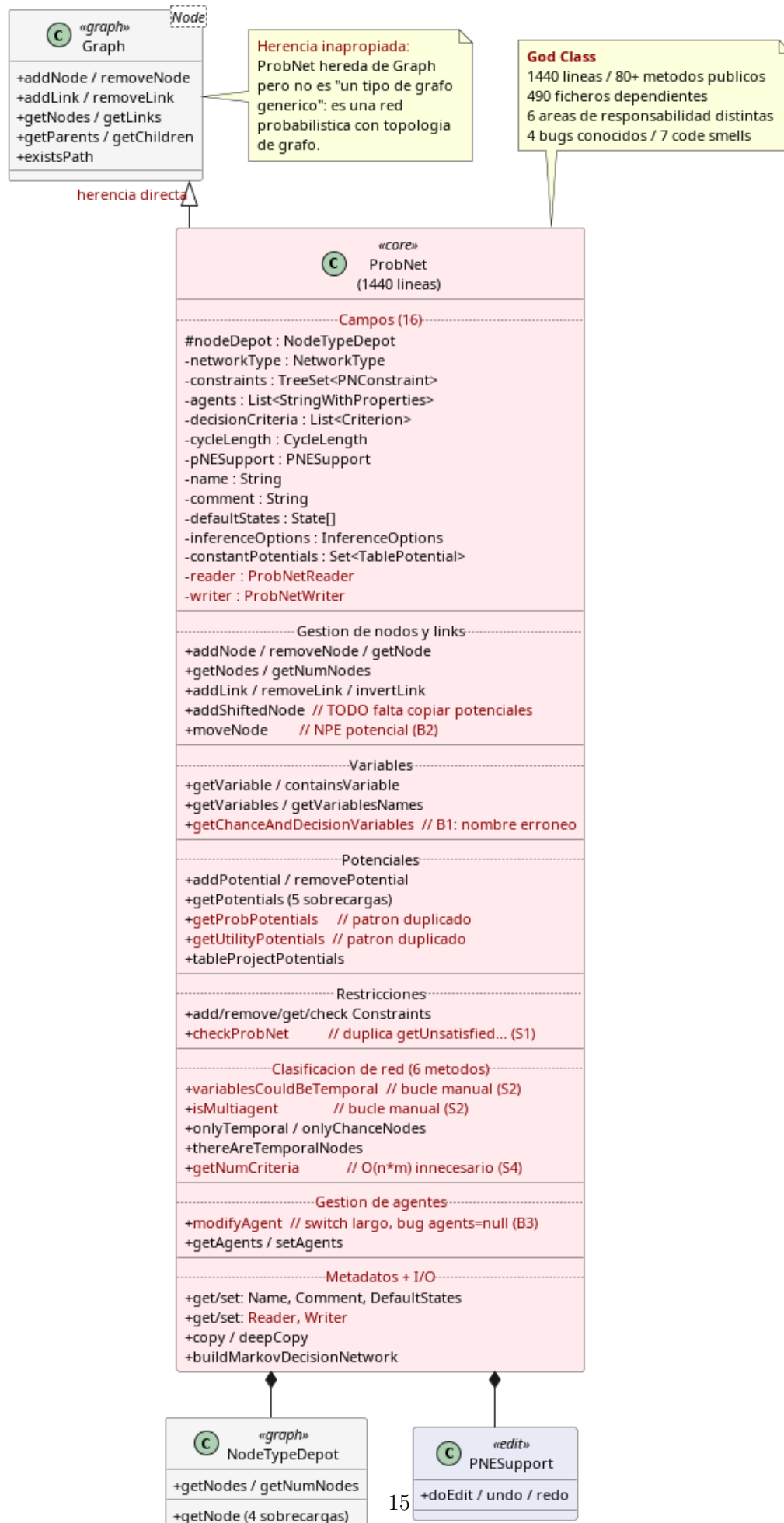


Figura 7: Estructura interna de ProbNet al inicio del rediseño.

5.2. Fase 1: corregir bugs y simplificaciones

Los cuatro bugs y los siete code smells se resolvieron sin modificar la API pública.

Bug B1 — Renombrar `getChanceAndDecisionVariables()`. El análisis del uso en el proyecto confirmó que los tipos `SV_PRODUCT/SV_SUM` debían incluirse, porque los llamadores esperaban ese comportamiento. El problema era el nombre: se renombró a `getNonUtilityVariables()`, que describe con precisión lo que el método hace. Se conservó el nombre antiguo como método `@Deprecated` para compatibilidad.

Bug B2 — Null-check en `moveNode()`. Se añadió una comprobación explícita: si `getNode(name)` devuelve `null`, el método ignora ese nombre en lugar de lanzar una excepción.

Bug B3 — Inicializar `agents` con lista vacía. Se eliminó la asignación `agents = null` en `modifyAgent(REMOVE)`. La lista se inicializa como vacía desde el principio y nunca se asigna `null`. Esto garantiza que `getAgents()` siempre devuelve una lista (posiblemente vacía), nunca `null`.

Bug B4 — `addShiftedNode()` sin potenciales. La resolución queda pendiente porque requiere decidir qué comportamiento esperan los llamadores: ¿copiar los potenciales del nodo original, o crear un potencial uniforme por defecto? Se añadió un test que documenta el comportamiento actual y el comentario `@ToCheck`.

Code smells S1–S7.

- S1: `checkProbNet()` reducido a `getUnsatisfiedConstraints().isEmpty()` (DRY).
- S2: `variablesCouldBeTemporal()` e `isMultiagent()` usan ahora `hasConstraintOfClass()`.
- S3: patrón duplicado resuelto al extraer `ProbNetPotentialQueries` (§5.7).
- S4: `getNumCriteria()` usa ahora un `TreeSet`.
- S5: eliminado el código comentado y convertida la cadena `if` en `switch` en `addNodeConsistently()`.
- S6: `toString()` movido al final del fichero.
- S7: `reader/writer` incluidos en `NetworkMetadata` (§5.3) como solución intermedia.

El listado 9 ilustra la corrección del smell S1.

```
1 // ANTES: 8 líneas duplicando la logica:
2 public boolean checkProbNet() {
3     for (PNConstraint constraint : constraints) {
4         if (!constraint.isMetBy(this)) { return false; }
5     }
6     return true;
7 }
8
9 // DESPUES: 1 línea que reutiliza el metodo existente:
10 public boolean checkProbNet() {
11     return getUnsatisfiedConstraints().isEmpty();
12 }
```

Listado 9: S1: `checkProbNet()` delega en `getUnsatisfiedConstraints()` (DRY).

5.3. Extracción de `NetworkMetadata`

Motivación (SRP, §1.1; Extract Class, §1.3; Value Object, §1.3): `ProbNet` almacenaba directamente once campos de metadatos: nombre, comentario, si mostrar el comentario al abrir, estados por defecto, criterios de decisión, agentes, longitud de ciclo, opciones de inferencia, propiedades adicionales de formato, `reader` y `writer`. Estos datos no tienen comportamiento de

dominio propio y cambiarán cuando cambie la interfaz de usuario o el formato de fichero, no cuando cambie la lógica de inferencia.

Decisión: se extrajeron los 11 campos a un objeto valor `NetworkMetadata` que `ProbNet` posee como campo privado. `ProbNet` conserva todos sus *getters* y *setters* públicos, delegando ahora en `NetworkMetadata`.

```

1 public class ProbNet extends Graph<Node> {
2     private String name;
3     private String comment = "";
4     private boolean showCommentWhenOpening = false;
5     private State[] defaultStates = { ... };
6     private InferenceOptions inferenceOptions = new InferenceOptions();
7     private CycleLength cycleLength = new CycleLength();
8     // ... 5 campos mas de metadatos ...
9
10    public String getName() { return name; }
11    public void setName(String name) { this.name = name; }
12    // ...
13 }

```

Listado 10: Antes: campos de metadatos directamente en `ProbNet`.

```

1 public class ProbNet implements PotentialNetwork {
2     private final NetworkMetadata metadata = new NetworkMetadata();
3     // campos de dominio: grafo, restricciones, potenciales...
4
5     public String getName() { return metadata.getName(); }
6     public void setName(String name) { metadata.setName(name); }
7     public String getComment() { return metadata.getComment(); }
8     public void setComment(String c) { metadata.setComment(c); }
9     // ... todos los demas getters/setters delegan igual ...
10 }

```

Listado 11: Después: metadatos delegados a `NetworkMetadata`.

El efecto en los campos de `ProbNet` fue inmediato: de **16 campos** pasó a **7**. Los 11 campos de metadatos fueron reemplazados por un único campo `metadata`.

La clase `NetworkMetadata` resultante agrupa:

Campo	Tipo
name	String
comment	String
showCommentWhenOpening	boolean
defaultStates	State[]
agents	List<StringWithProperties>
decisionCriteria	List<StringWithProperties>
cycleLength	CycleLength
inferenceOptions	InferenceOptions
additionalProperties	Map<String, String>
reader	ProbNetReader
writer	ProbNetWriter

¿Por qué los campos `reader/writer` no se movieron fuera de `ProbNet`? La solución correcta al smell S7 sería mover estos campos al controlador de ficheros de la GUI. Sin embargo, hacerlo requiere un análisis de impacto en los módulos `gui` e `io` que va más allá

del alcance de esta refactorización. Como solución intermedia, los campos se incluyen en `NetworkMetadata`, donde al menos quedan claramente identificados como “datos de sesión”, no como comportamiento de dominio. La migración completa está documentada como tarea pendiente.

5.4. Extracción de `ProbNetCopier`

Motivación (SRP, §1.1; Extract Class, §1.3): `ProbNet` contenía dos métodos de copia, `copy()` y `deepCopy()`, con implementaciones de 40 y 50 líneas respectivamente. La lógica es compleja: iterar nodos, arcos, potenciales y restricciones, y decidir qué se comparte por referencia (copia superficial) y qué se instancia de nuevo (copia profunda). Esta lógica no pertenece al modelo de dominio: el modelo de la red no tiene por qué saber cómo preparar una copia de sí mismo para inferencia.

Decisión: se extrajo la lógica de copia a `ProbNetCopier` (visibilidad de paquete, no pública). Los métodos `copy()` y `deepCopy()` de `ProbNet` permanecen en la API pública como delegaciones:

```
1 // ProbNet.java (API publica inalterada):
2 public ProbNet copy() {
3     return ProbNetCopier.shallowCopy(this);
4 }
5
6 public ProbNet deepCopy() {
7     return ProbNetCopier.deepCopy(this);
8 }
9
10 // ProbNetCopier.java (logica compleja, fuera del dominio):
11 final class ProbNetCopier {
12     private ProbNetCopier() {} // no instanciable
13
14     static ProbNet shallowCopy(ProbNet original) {
15         // Copia la estructura compartiendo referencias a
16         // Variables y Potentials (semantica "copy"):
17         // - nodos nuevos con las mismas Variables (compartidas)
18         // - arcos nuevos
19         // - potenciales compartidos (no clonados)
20         // - restricciones compartidas
21         // ... 40 líneas de logica ...
22     }
23
24     static ProbNet deepCopy(ProbNet original) {
25         // Crea nuevas instancias de todos los objetos mutables:
26         // - nodos nuevos con nuevas Variables
27         // - nuevos potenciales clonados
28         // - nuevas restricciones
29         // ... 50 líneas de logica ...
30     }
31 }
```

Listado 12: Extracción de la lógica de copia a `ProbNetCopier`.

5.5. Extracción de `ProbNetClassifier`

Motivación (SRP, §1.1; Extract Class, §1.3): `ProbNet` contenía seis métodos cuyo único propósito era clasificar la red según sus restricciones y contenido: `variablesCouldBeTemporal()`, `isMultiagent()`, `onlyTemporal()`, `onlyChanceNodes()`, `thereAreTemporalNodes()` y `getNumCriteria()`. Estos métodos son *consultas puras*: no modifican la red. Su único consumidor relevante era la GUI y los algoritmos de configuración.

Por qué como clase estática: los métodos de clasificación no necesitan acceder a ningún estado mutable de ProbNet; solo consultan restricciones y nodos. Una clase de utilidad estática (como VariableTypeConverter y VariableStateOperations) es la opción más honesta: hace explícito que no hay estado y que los métodos son funciones puras.

Retrocompatibilidad: ProbNet conserva los seis métodos de instancia como delegaciones hacia ProbNetClassifier.xxxx(this).

```

1 // En ProbNet.java (retrocompatibilidad garantizada):
2 public boolean variablesCouldBeTemporal() {
3     return ProbNetClassifier.variablesCouldBeTemporal(this);
4 }
5 public boolean isMultiagent() {
6     return ProbNetClassifier.isMultiagent(this);
7 }
8 // ... otros cuatro metodos analogos ...

```

Listado 13: Delegaciones en ProbNet después de la extracción.

```

1 public final class ProbNetClassifier {
2
3     private ProbNetClassifier() {} // no instanciable
4
5     public static boolean variablesCouldBeTemporal(ProbNet net) {
6         return !net.hasConstraintOfClass(OnlyAtemporalVariables.class);
7     }
8
9     public static boolean isMultiagent(ProbNet net) {
10        return !net.hasConstraintOfClass(OnlyOneAgent.class);
11    }
12
13    public static boolean onlyTemporal(ProbNet net) {
14        return net.hasConstraintOfClass(OnlyTemporalVariables.class);
15    }
16
17    public static boolean onlyChanceNodes(ProbNet net) {
18        return net.hasConstraintOfClass(OnlyChanceNodes.class);
19    }
20
21    public static boolean thereAreTemporalNodes(ProbNet net) {
22        return net.getNodes().stream()
23            .anyMatch(n -> n.getVariable().isTemporal());
24    }
25
26    public static int getNumCriteria(ProbNet net) {
27        // TreeSet deduplica en O(n log n), no O(n*m) como antes
28        Set<String> names = new TreeSet<>();
29        net.getPotentials(PotentialRole.UTILITY)
30            .forEach(p -> names.add(p.getCriterion().toString()));
31        return names.size();
32    }
33 }

```

Listado 14: Implementación completa de ProbNetClassifier.

5.6. Extracción de ProbNetAgentManager

Motivación (SRP, §1.1; Extract Class, §1.3): la gestión de agentes en ProbNet se realizaba mediante un método modifyAgent() que implementaba cuatro operaciones (ADD, REMOVE, RENAME, UP/DOWN) en un switch largo, con lógica de limpieza de referencias en nodos al

eliminar un agente. Esta responsabilidad es completamente ajena a la lógica de inferencia y solo la usa la GUI. Además, el bug B3 vivía exactamente en ese `switch`.

La extracción a `ProbNetAgentManager` resolvió el bug B3 con naturalidad: al aislar la lógica de agentes en su propia clase, el invariante “la lista de agentes nunca es `null`” queda claramente establecido.

```

1 public final class ProbNetAgentManager {
2
3     private ProbNetAgentManager() {}
4
5     public static void modifyAgent(ProbNet net,
6                                   StateAction action,
7                                   String agentName,
8                                   Object[][] dataTable) {
9         List<StringWithProperties> agents = net.getAgents();
10        switch (action) {
11            case ADD -> {
12                if (agents == null) agents = new ArrayList<>();
13                agents.add(new StringWithProperties(agentName));
14                net.setAgents(agents);
15            }
16            case REMOVE -> {
17                // ANTES: agents = null <-- causaba NPE posterior
18                // AHORA: eliminar el agente y limpiar sus nodos
19                agents.removeIf(a -> a.getString().equals(agentName));
20                for (Node node : net.getNodes(NodeType.DECISION)) {
21                    if (agentName.equals(node.getAgent())) {
22                        node.setAgent(null);
23                    }
24                }
25                net.setAgents(agents); // lista vacia, nunca null
26            }
27            case RENAME -> {
28                agents.stream()
29                    .filter(a -> a.getString().equals(agentName))
30                    .findFirst()
31                    .ifPresent(a -> a.setString(
32                        (String) dataTable[0][1]));
33                net.setAgents(agents);
34            }
35            // ... casos UP, DOWN ...
36        }
37    }
38 }

```

Listado 15: Bug B3 corregido en `ProbNetAgentManager`.

5.7. Extracción de `ProbNetPotentialQueries`

Motivación (SRP, §1.1; DRY, §1.1; Extract Class, §1.3): `ProbNet` contenía cuatro métodos de consulta complejas sobre potenciales: `getProbPotentials()`, `getUtilityPotentials()`, `tableProjectPotentials()` y `buildMarkovDecisionNetwork()`.

Los dos primeros violaban DRY (smell S3): mismo algoritmo, diferente predicado. Además, estos métodos realizan trabajo más allá del simple acceso a datos (construyen objetos, filtran, transforman): ese nivel de lógica no pertenece al contenedor de la red.

Decisión: se extrajeron los cuatro métodos a `ProbNetPotentialQueries`, unificando el patrón duplicado en un método privado parametrizado:

```

1 public final class ProbNetPotentialQueries {
2
3     private ProbNetPotentialQueries() {}
4
5     // Un unico metodo privado con el patron comun:
6     private static List<Potential> getPotentialsByRole(
7         ProbNet net, Variable variable, PotentialRole role) {
8         List<Potential> result = new ArrayList<>();
9         Node varNode = net.getNode(variable);
10        if (varNode == null) return result;
11        Set<Node> neighbors = new HashSet<>(net.getNeighbors(varNode));
12        neighbors.add(varNode);
13        for (Node neighbor : neighbors) {
14            for (Potential p : neighbor.getPotentials()) {
15                if (p.getPotentialRole() == role
16                    && p.getVariables().contains(variable)) {
17                    result.add(p);
18                }
19            }
20        }
21        return result;
22    }
23
24    // Los dos metodos publicos son ahora llamadas de dos lineas:
25    public static List<Potential> getProbPotentials(
26        ProbNet net, Variable variable) {
27        return getPotentialsByRole(
28            net, variable, PotentialRole.CONDITIONAL_PROBABILITY);
29    }
30
31    public static List<Potential> getUtilityPotentials(
32        ProbNet net, Variable variable) {
33        return getPotentialsByRole(
34            net, variable, PotentialRole.UTILITY);
35    }
36
37    public static List<Potential> tableProjectPotentials(
38        ProbNet net, List<Variable> vars, ...) {
39        // ... logica de proyeccion ...
40    }
41
42    public static ProbNet buildMarkovDecisionNetwork(ProbNet net) {
43        // ... construccion de la MDN para POMDP/DecPOMDP ...
44    }
45 }

```

Listado 16: Unificación del patrón duplicado en ProbNetPotentialQueries.

5.8. El cambio más profundo: de herencia a composición

Motivación (composición frente a herencia, §1.1; ISP, §1.1): la relación ProbNet extends Graph<Node> era el problema de acoplamiento más profundo de la clase. Significaba que ProbNet exponía toda la API interna del grafo genérico (addVertex, removeVertex y otras operaciones de bajo nivel de JGraphT) mezclada con la API de dominio. No había forma de que un algoritmo de inferencia declarase que solo necesitaba “consultar el grafo” sin depender de toda la clase concreta.

Decisión: dos interfaces y composición interna. Se introdujeron:

- GraphNetwork: vista de solo lectura de la topología del grafo y las variables. Define los métodos de consulta de grafo sin exponer mutaciones.

- `PotentialNetwork` extends `GraphNetwork`: extiende `GraphNetwork` con acceso a potenciales. Es la interfaz mínima que necesita un algoritmo de inferencia completo.

La Figura 8 ilustra la jerarquía.

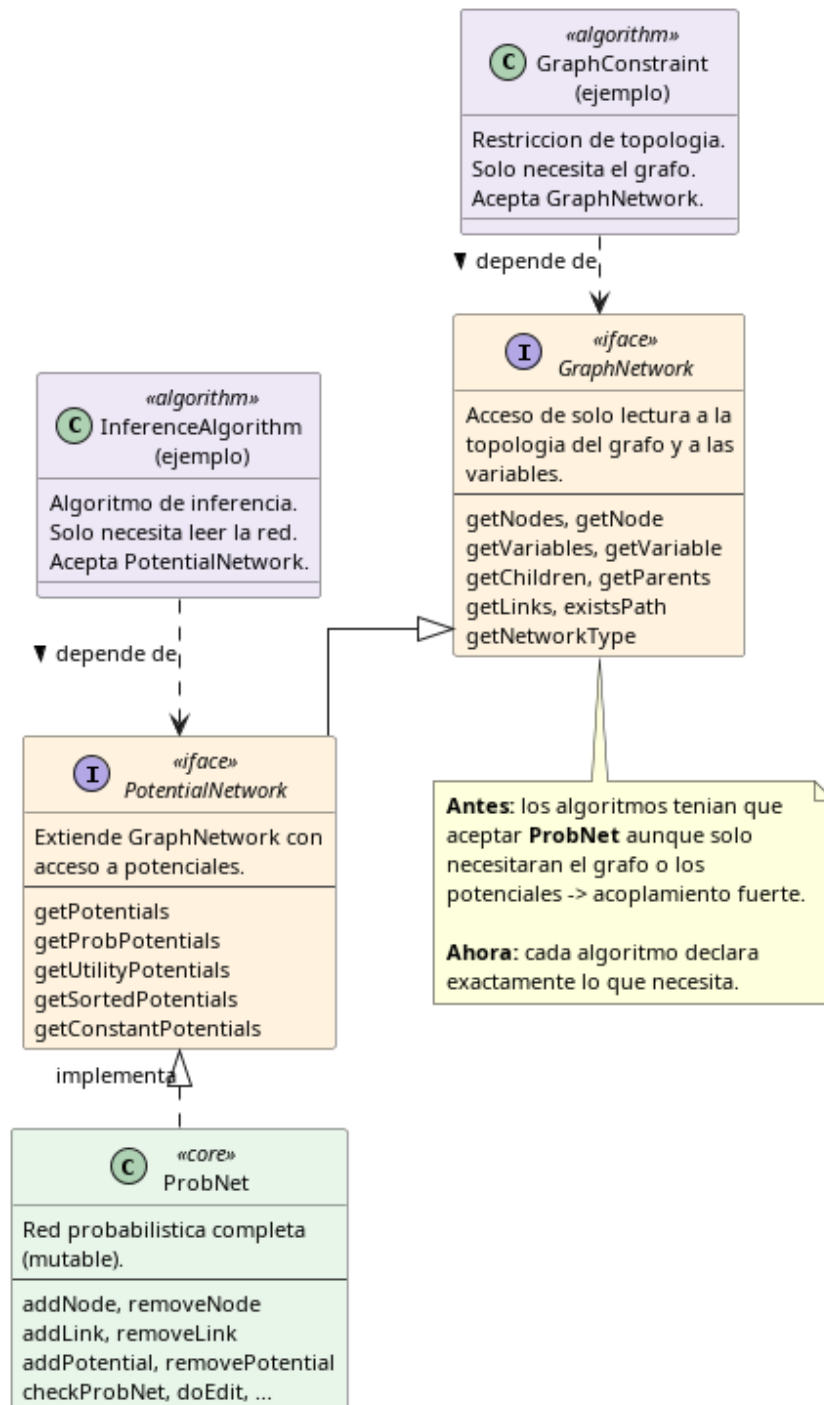


Figura 8: Jerarquía de interfaces. Los algoritmos que solo necesitan el grafo dependen de `GraphNetwork`; los que necesitan potenciales, de `PotentialNetwork`. `ProbNet` implementa ambas.

`ProbNet` pasa de `extends Graph<Node>` a `implements PotentialNetwork` y almacena el grafo como campo privado:

```

1 // ANTES:
2 public class ProbNet extends Graph<Node> implements ... {
3     // hereda addVertex, removeVertex y toda la API de Graph
4     // el grafo es ProbNet misma; no hay forma de ocultarlo
5 }
6
7 // DESPUES:
8 public class ProbNet implements PotentialNetwork, ... {
9     // el grafo es un detalle de implementacion privado:
10    private final Graph<Node> graph = new Graph<>();
11
12    // ProbNet expone solo las operaciones con semantica de red:
13    @Override
14    public List<Node> getNodes() {
15        return graph.getNodes(); // delega internamente
16    }
17
18    @Override
19    public List<Node> getChildren(Node node) {
20        return graph.getSuccessors(node);
21    }
22
23    @Override
24    public List<Node> getParents(Node node) {
25        return graph.getPredecessors(node);
26    }
27    // ...
28 }

```

Listado 17: Cambio de herencia a composición en ProbNet.

Efecto en las restricciones. Las 35 clases de restricción (PNConstraint) declaraban `checkProbNet(ProbNet net, ...)` y pasaron a `checkProbNet(GraphNetwork net, ...)`. Esto redujo el acoplamiento de 35 clases de una sola vez:

```

1 // ANTES: cada restriccion dependia de la clase concreta
2 public class NoCycleConstraint extends PNConstraint {
3     @Override
4     public boolean checkProbNet(ProbNet net, List<Variable> vars) {
5         // ...usa net.getNodes() y net.existsPath()...
6     }
7 }
8
9 // DESPUES: solo necesita la interfaz de topologia
10 public class NoCycleConstraint extends PNConstraint {
11     @Override
12     public boolean checkProbNet(GraphNetwork net, List<Variable> vars) {
13         // la interfaz garantiza exactamente lo que se usa
14     }
15 }

```

Listado 18: Restricciones desacopladas de ProbNet.

Las cinco restricciones que necesitaban ProbNet concreto. Cinco restricciones (ModelNetworkConstraint, ProperUtilityPotentials, entre otras) usaban operaciones que GraphNetwork no expone, como comprobar potenciales. Se decidió mantener un cast interno a ProbNet dentro de estas restricciones en lugar de ampliar la interfaz con operaciones que la mayoría no necesita. Es una solución pragmática que evita comprometer el diseño general por casos excepcionales.

6. El resultado

6.1. La nueva arquitectura

La Figura 9 muestra la arquitectura de ProbNet y sus clases satélite después del rediseño.

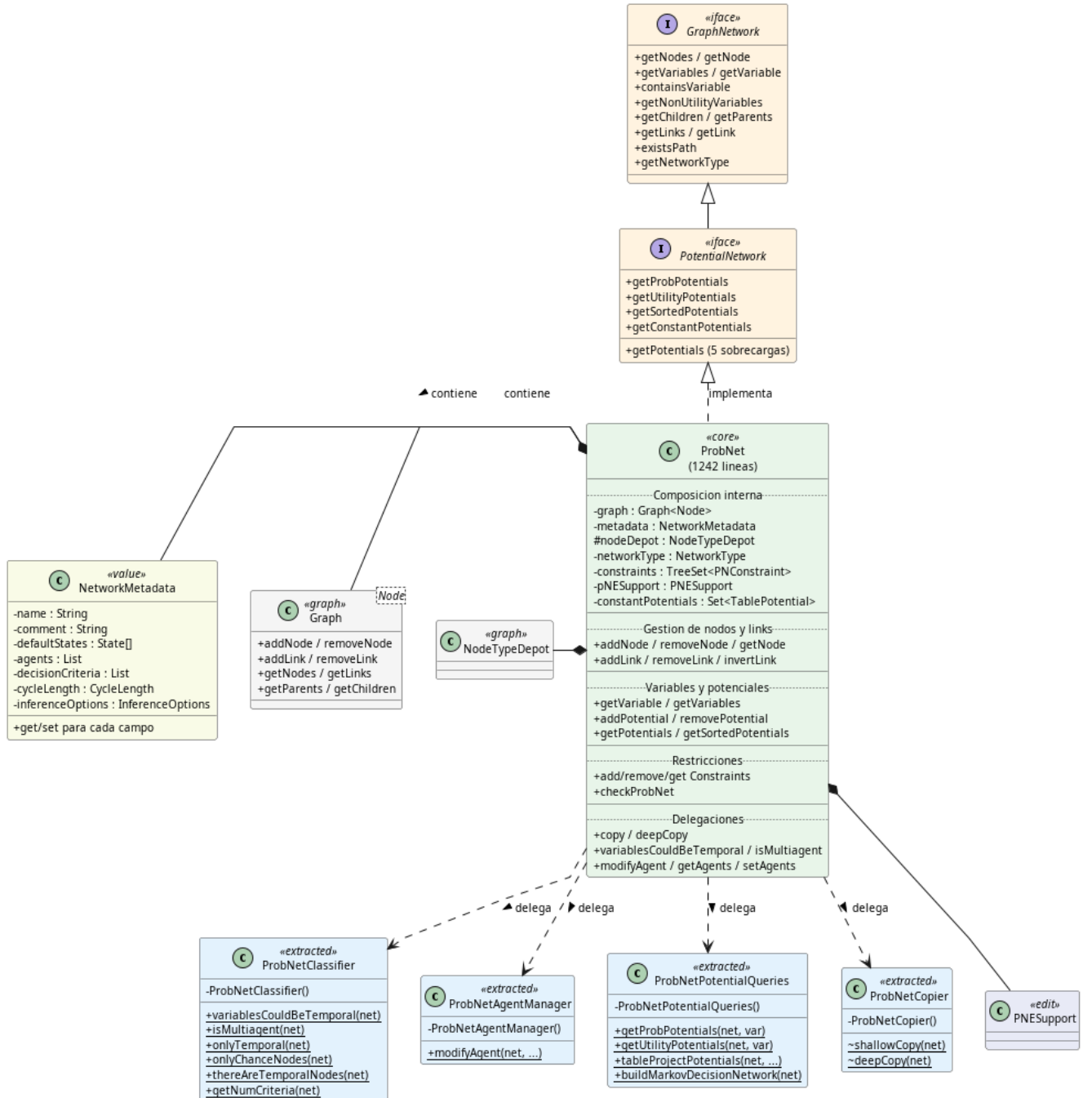


Figura 9: Arquitectura de ProbNet después del rediseño (1242 líneas). En azul, las clases extraídas. En naranja, las nuevas interfaces. En verde y gris, los componentes de composición interna.

6.2. Las clases extraídas

Clase	Líneas	Responsabilidad
GraphNetwork	92	Interfaz: vista de solo lectura del grafo.
PotentialNetwork	47	Interfaz: extiende GraphNetwork con potenciales.
NetworkMetadata	93	Objeto valor: todos los metadatos descriptivos.
ProbNetCopier	168	Copia superficial y profunda de la red.
ProbNetClassifier	90	Consultas de clasificación (6 métodos estáticos).
ProbNetAgentManager	87	Gestión de agentes (ADD/REMOVE/RENAME/UP/DOWN).
ProbNetPotentialQueries	134	Consultas complejas sobre potenciales.
Total nuevas clases	711	

6.3. Métricas comparadas

Métrica	Antes	Después	Cambio
Líneas de ProbNet.java	1 440	1 242	−14 %
Campos en ProbNet	16	7	−56 %
Bugs conocidos	4	1*	−75 %
Code smells	7	0	−100 %
Restricciones acopladas a ProbNet	35	0	todas usan GraphNetwork
Clases nuevas en el paquete network	—	+7	
Líneas totales del paquete (redistribuidas)	1 440	~1 953	

* Bug B4 (addShiftedNode() sin potenciales) permanece documentado con @ToCheck pendiente de decisión sobre el comportamiento correcto.

La reducción de ProbNet de 1440 a 1242 líneas puede parecer pequeña comparada con las 711 líneas de las siete clases nuevas. El código total del paquete creció. Esto es el resultado esperado cuando se hace *explícito* el trabajo que una clase hacía de forma *implícita*: la God Class no era eficiente en términos de líneas; era pequeña *aparentemente* porque concentraba complejidad. Al redistribuirla, la complejidad se hace visible y, sobre todo, se hace *dirigible*: cada clase nueva puede mantenerse, probarse y entenderse de forma independiente.

6.4. Estado de ejecución de las fases

Fase	Descripción	Riesgo	Estado
1	Corrección de bugs B1–B3 y smells S1–S7	Bajo	Completada
	Extracción de NetworkMetadata	Bajo	Completada
	Extracción de ProbNetCopier	Bajo	Completada
3	Extracción de ProbNetClassifier	Medio	Completada
4	Extracción de ProbNetAgentManager	Bajo	Completada
5	Extracción de ProbNetPotentialQueries	Medio	Completada
	Interfaces GraphNetwork/PotentialNetwork	Alto	Completada
	Adaptar 35 restricciones a GraphNetwork	Medio	Completada
–	Bug B4 (addShiftedNode sin potenciales)	Bajo	Pendiente
–	Mover reader/writer a la GUI	Alto	Pendiente

Las dos tareas pendientes son independientes del resto del rediseño. La primera requiere consultar el comportamiento esperado en la GUI. La segunda requiere un análisis de impacto en los módulos `gui` e `io`.

6.5. Qué no cambió: la API pública

Los **490** **ficheros** que referencian `ProbNet` no necesitaron ninguna modificación para que los tests siguieran pasando. Esto fue posible gracias al patrón de delegación retrocompatible aplicado sistemáticamente en cada extracción (§4).

La migración progresiva hacia las nuevas clases e interfaces es posible pero no urgente. Un llamador puede pasar de `net.isMultiagent()` a `ProbNetClassifier.isMultiagent(net)` cuando lo considere conveniente, o puede usar `GraphNetwork` en lugar de `ProbNet` en la firma de un nuevo método. No hay presión de tiempo.

7. Reflexión sobre el proceso

7.1. Lo que facilitó la refactorización

El patrón *Extract Class* tenía precedente en el proyecto. `VariableTypeConverter` (extraída de `Node`) y `VariableStateOperations` (extraída de `Variable`) eran refactorizaciones del mismo tipo, hechas antes con éxito. Tener un precedente reduce la resistencia al cambio: la propuesta no era especulativa, era una aplicación más del mismo patrón.

La retrocompatibilidad como invariante. Establecer desde el principio que “la API pública no se rompe” simplificó la toma de decisiones. Ante cualquier disyuntiva, la pregunta era: ¿rompe esto alguna signatura pública existente? Si la respuesta era sí, la opción quedaba descartada.

La fase de diagnóstico por separado. Dedicar tiempo al análisis *antes* de cambiar código produjo un diagnóstico con bugs numerados y smells catalogados. Cuando durante la ejecución surgía la duda de si corregir el bug B3 dentro de la extracción o antes, había un documento de referencia.

7.2. Lo que fue más difícil de lo esperado

El cambio de herencia a composición. Las 35 restricciones tuvieron que adaptarse. La adaptación era mecánica (cambiar el tipo del parámetro), pero ninguna herramienta de refactorización automática del IDE pudo hacerlo de una sola vez, porque cinco restricciones necesitaban un cast interno a `ProbNet` para operaciones que `GraphNetwork` no expone.

La frontera entre `GraphNetwork` y `PotentialNetwork`. Decidir qué métodos pertenecen a cada interfaz requirió revisar cuáles usa cada algoritmo de restricción. La línea divisoria no era obvia a priori: métodos como `getNonUtilityVariables()` mezclan topología (qué nodos hay) con rol (cuáles son de utilidad).

Algunas correcciones son más naturales dentro de una extracción. El bug B3 se corrigió con más claridad dentro de `ProbNetAgentManager` porque la extracción forzó un razonamiento explícito sobre el invariante de la lista de agentes. Corregirlo antes, dentro de la clase grande, habría funcionado, pero habría sido más fácil introducir el mismo error de nuevo.

7.3. Lecciones para futuros procesos de refactorización

1. **Diagnosticar antes de cambiar.** Un diagnóstico escrito con bugs numerados, smells catalogados y cobertura medida acelera la ejecución y facilita las revisiones.
2. **Los tests son la red de seguridad.** Antes de extraer código, hay que tener tests que lo cubran. Sin tests, la refactorización es potencialmente destructiva.
3. **Pequeños pasos verificables.** Cada fase dejó el sistema compilando y con todos los tests pasando. Esto permitió detectar errores inmediatamente después del cambio que los causó.
4. **La retrocompatibilidad tiene un coste que vale la pena.** Mantener delegaciones en ProbNet añade líneas que en un diseño ideal no existirían. En un sistema con 490 dependencias, el coste de esas líneas es mucho menor que el de migrar 490 ficheros de una sola vez.
5. **La deuda técnica no resuelta debe documentarse.** Las dos tareas pendientes (B4 y el smell S7) están documentadas con el contexto necesario para quien las aborde en el futuro. Dejarlas sin resolver fue una decisión deliberada, no un olvido.

Referencias

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2019.
- [2] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [3] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- [4] A. Hunt, D. Thomas, *The Pragmatic Programmer*, Addison-Wesley, 1999.